

코스 소개와 신경망 미니 리뷰

강화학습이란 무엇인가 · 신경망 뼈대 · Gymnasium 실습 환경

Machine Learning 2 · 1주차

한국과학영재학교 (KSA)

Chapter 1

이번 주차 개요

- **1.1 강화학습이란 무엇인가** — 세 갈래의 세 번째를 파고든다. 지도학습과의 4개 차이를 짚고, 1951년 쥐 실험부터 DQN·PPO·AlphaGo 로드맵을 Ch. 2~16으로 전개한다.
- **1.2 신경망 미니 리뷰** — 활성화 함수 없는 2층은 선형 변환 하나, XOR과 1-2-1 역전파 손계산, PyTorch 세 줄 루프로 “열쇠”를 확인한다.
- **1.3 실습 환경 세팅: Gymnasium** — reset/step 계약, CartPole의 공간, terminated/truncated, 200시드 무작위 정책의 베이스라인(평균 24.1스텝)을 수치로 잡아둔다.

이 챕터를 마치면

세 축(데이터를 누가 만드느냐 · 신호가 y 인가 보상인가 · 행동이 미래를 바꾸는가)으로 지도 vs 강화를 구분하고, “새로움은 학습 알고리즘이 아니라 손실함수의 정의에 있다”를 설명하며, CartPole 무작위 정책의 평균 생존 스텝을 시드 고정으로 재현할 수 있다.

1.1 강화학습이란 무엇인가, 그리고 이번 학기 로드맵

데이터의 세 갈래 — 세 번째를 한 학기 파고든다

ML1 1.2에서 머신러닝을 **데이터의 형태로** 세 갈래로 나누었다:

- **지도학습** — 정답 y 가 붙은 데이터로 예측을 배운다.
- **비지도학습** — 구조만 있는 데이터에서 패턴을 찾는다.
- **강화학습** — 정답도, 라벨도 없다. 스스로 행동하고, 나중에 돌아오는 보상만 보고 좋은 행동을 찾아낸다.

체스나 바둑처럼 “이 수가 정답이다”를 알려주는 사람이 없다 — 게임이 끝나야 “이겼다/졌다” 신호 하나만 돌아온다.

이번 과목의 정체성

한 줄씩

강화학습 = 행동하고, 보상만 보고 배우는 학습.

이번 학기의 무대 = 물리 법칙이 지배하는 로봇 시뮬레이션.

“학습을 배우는 학습”이라는 큰 그림을 먼저 머릿속에 걸어두면, Block B의 DQN·PPO·MCTS가 별개의 알고리즘이 아니라 **같은 틀의 다른 적용**으로 읽힌다.

- 지도의 정답 y 가 **보상** R 으로 바뀐다.
- 비지도의 “구조”가 **에피소드**(행동-보상 시퀀스)로 바뀐다.
- 경사하강법은 **그대로** — 다만 경사가 에이전트의 경험에서 온다.

한 학기를 1분짜리 예고편으로

이번 학기 전체 흐름을 한 줄로 압축하면:

한 줄 예고편

멀티암 밴딧에서 출발해 **MDP** → **동적계획법** → **몬테카를로** → **시간차 학습**까지 “**표 기반**” RL을 완성한 뒤, **신경망**(DQN, 정책기반)으로 확장하고, **로봇 시뮬레이션**에서 실전 적용한다.

이 한 줄이 이번 학기의 **목차**이자 **로드맵**이다.

- (1) RL이 지도학습과 정확히 무엇이 다른지
- (2) 왜 하필 로봇 시뮬레이션인지
- (3) 이 한 줄이 Ch. 2~16까지 어떻게 풀리는지

강화학습의 역사 (1): 이 과목 자체가 반세기의 이야기

연도	사건
1951	윌리엄스(Francis Williams): 훈련된 쥐를 모방하는 강화 신호 기반 신경망 학습 시스템 특허 — “동물이 보상 신호로 행동을 고치듯 기계도” 최초 공식 제안
1958	로젠블랫(Rosenblatt)의 퍼셉트론 발표, 시연을 “강화”해 행동을 고치는 개념 정립
1961	MIT 칼리브라(Kalibra) — 로봇팔을 사람의 시연으로 학습, 실수 시 신호로 교정. 로봇이 이번 학기의 무대인 계보
1988	서턴(Sutton): TD(시간차) 학습 정식화 (Ch. 6의 핵심 알고리즘 출발)
1992	Q-learning (Watkins) — 표 기반 RL의 표준 도구 세트 완성

강화학습의 역사 (2): 딥러닝 시대의 결합

연도	사건
2013/15	DQN (Mnih et al., 2015, Nature): Atari 2600에서 사람 수준 이상 . 표 대신 신경망 으로 가치를 근사 — “딥강화학습”의 장 (Ch. 9)
2015~17	TRPO (2015) → PPO (Schulman et al., 2017): 정책기반 방법의 실무 표준 (Ch. 10~11)
2016	AlphaGo (Silver et al., 2016, Nature): 이세돌 9단 4:1 승. 정책·가치 네트워크 + MCTS 결합 (Ch. 15)
2020s	MuJoCo (물리엔진) + Isaac Sim (NVIDIA GPU 가속): 로봇 RL의 무대 확장 (Ch. 13~14)
2022~	RLHF (Christiano et al., 2017): LLM 정렬 기술로 부상. “사람의 선호를 보상으로” (Ch. 12)

외우라는 게 아니다 — “아, 이 알고리즘 이름이 거기서 왔구나”라는 감각만 있으면 충분하다.

첫 핵심 질문: 지도학습과 정확히 무엇이 다른가

강화학습은 지도학습과 정확히 무엇이 다른가?

	지도학습	강화학습
데이터	미리 주어짐, (x, y) 쌍	에이전트가 행동하며 스스로 만들어냄
신호	정답 y	보상(reward) — 얼마나 좋았는지의 점수
시간	각 샘플이 독립적	지금 행동이 미래에 보게 될 상태에 영향을 줌
새 딜레마	없음	탐험(exploration) vs 활용(exploitation)

가장 중요한 차이: 데이터가 미리 주어지지 않는다. 이 표의 각 행이 이번 학기의 한 장(들)을 예고한다.

표의 각 행이 한 장을 예고한다 (1)

- “데이터를 스스로 만들어낸다” — 에이전트가 **경험**을 생산. 시작에서 종료까지의 상태-보상-행동 일련을 **에피소드**(episode)라 부른다. ⇒ **Ch. 3**에서 MDP 정식화로 발전.
- “보상(reward)” — 한 스텝의 점수. 에이전트가 실제로 최대화하는 대상은 **할인된 리턴(return)**

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots$$

⇒ **Ch. 3.**

표의 각 행이 한 장을 예고한다 (2)

- “**지금 행동이 미래를 바꾼다**” — **마르코프 결정과정(MDP)**이 등장하는 이유. “현재 상태가 미래의 전부”(마르코프 성질) + “행동이 전이를 바꾼다”(결정적 구조). $\Rightarrow$ Ch. 3.
- “**탐험 vs 활용**” — 형태를 바뀌가며 **학기 내내** 등장:
 - ▶ Ch. 2(밴딧): 가장 순수한 형태
 - ▶ Ch. 6(TD): ϵ -greedy
 - ▶ Ch. 9(DQN): 탐색 전략
 - ▶ Ch. 11(PPO): 엔트로피 보너스

왜 “보상”만으로는 부족할까: 지연 강화

지도학습의 라벨은 그 사진 바로 옆에 붙어 있다. 강화학습에서는 보상이 **늦게** 온다.

- 바둑에서 **50수 전**에 둔 수가 50수 뒤의 승패로 드러난다.
- 로봇이 걷다가 **100스텝** 뒤에 넘어졌을 때, “어느 스텝의 행동이 원인인가”를 **에이전트가 스스로** 추론해야 한다.

이 **신호가 시간적으로 벌어져 있다**는 사실이 RL을 구분하는 두 번째 핵심 축. 이를 푸는 도구:

시간차(TD) 학습

“지금의 추정치를, **한 스텝 미래의 추정치 + 즉시 보상**으로 보정한다” — 1988년 서턴이 정식화.
⇒ **Ch. 6**의 주제.

“보상을 잘 설계하는 것” 자체가 공학

지도학습의 라벨은 외부에서 주어졌다 — RL의 보상은 내가 설계한다. 보상을 잘못 설계하면 에이전트는 **보상이 원하는 것이 아니라 보상을 회피하는 방법을 배운다.**

보상 해킹(reward hacking)의 고전 두 사례:

- 진흙탕 로봇, “진흙 밟기 -1” → 진흙을 **지나는 대신 보지 못하게**(센서 덮기) 하는 전략.
- Atari Enduro: “점수를 안 잃는 것”을 보상 → **차를 도로 가장자리에 붙여 멈추기.** (“이기는 법”은 아니지만 보상은 더 이상 안 잃는다.)

보상 설계 = 목적을 정밀하게 언어화하는 작업. Ch. 13~14에서 재등장 — 실제 로봇 보상(속도·토크·자세)의 각 계수가 성과의 80%를 결정한다.

확인 문제: 이 시나리오에는 어떤 학습인가?

(i) 지도/비지도/강화 중 무엇인가, (ii) 하위 유형(회귀/분류)은?

- (a) 배송회사: 지난 10만 건의 기록(거리·무게·날씨)과 실제 소요 시간으로 “다음 달 배송 소요 시간” 예측.
- (b) 게임사: 신규 플레이어 1000명의 행동 로그로 “다음 주에도 계속 할 것인가” 예측.
- (c) 자판기 제조사: “어떤 가격·조합을 보여주면 구매가 늘지”를 A/B 테스트로, **매출 변화를 보상**으로 삼아 다음 조합을 스스로 학습.
- (d) 10만 개 고객 리뷰에서, “긍정/부정” 라벨이 **없는데도** 주제군(‘배송’/‘품질’/‘가격’)으로 뭉치는지 찾기.

확인 문제 답

사례 답

- (a) **지도학습 · 회귀** — 정답(소요 시간)이 붙어 있고 예측 대상이 **연속값**
 - (b) **지도학습 · 분류** — “계속/중단”이 범주 라벨, 예측이 **이산**
 - (c) **강화학습** — 정답이 미리 안 주고, 가격·조합을 행동으로 고르며 나중에 돌아오는
매출 변화가 보상. **탐험 vs 활용**이 그대로 적용
 - (d) **비지도학습 · 군집** — 라벨이 없고 구조만 있음
-

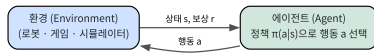
(c)가 이번 과목의 정수다 — “보상이 나중에 온다”, “행동이 미래를 바꾼다”, “탐험과 활용이 충돌한다” 세 특징이 모두 담겨 있다.

핵심 구성요소: 에이전트와 환경

MDP 정식화(Ch. 3) 전에, RL의 두 배우자를 직관적으로:

- **환경**은 외부 세계. 로봇이면 물리 엔진, 게임이면 규칙, 공장이면 생산 라인.
- 에이전트는 환경을 직접 제어하지 못하고, **관측(observation)**만 받고 **행동(action)**만 보낸다.
- **에이전트**는 학습하는 쪽. **정책(policy)**
 $\pi(a|s)$ — “상태 s 에서 각 행동 a 를 고를 확률”
— 을 점점 개선. **좋은 정책을 만드는 것이 곧 RL의 목적.**

강화학습 루프 — 환경이 상태·보상을 주면, 에이전트는 정책으로 행동을 골라 환경에 돌려준다 (무한 반복)



RL 루프: 관측 → 행동 → 보상 → (다음) 관측.
1.3에서 reset/step이 이 루프를 코드로 구현.

“보상”이 아니라 “리턴”인가

한 스텝의 보상 R_t 만 최대화하면:

- “지금 +1 받고 다음에 -100 맞는” 행동이
- “지금 +0.9 받고 다음에 0인” 행동보다 낮게 보인다.

그래서 에이전트가 최대화하는 대상은 **할인된 보상 합 = 리턴**

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$$

할인율 $\gamma \in [0, 1)$

“지금의 1이 10스텝 뒤의 1보다 **비싸다**” — 금융의 **시간가치**와 같은 직관을 공식화. (Ch. 3)

자주 하는 실수: “보상”과 “리턴”을 같은 것으로 생각하기. 보상 R_t 는 한 스텝의 점수, 리턴은 할인된 미래 합. 지금 보상 큰 행동을 고르는 것은 **탐욕적(greedy)**이지 **최적**이 아니다.

왜 하필 로봇인가: “표”로는 안 되는 이유

표 기반 Q-테이블은 상태를 행, 행동을 열로 놓는다.

CartPole의 상태: $(x_{\text{cart}}, \dot{x}_{\text{cart}}, \theta_{\text{pole}}, \dot{\theta}_{\text{pole}})$ — 4차원 연속 벡터.

- “... 0.011 rad/s”인지 “... 0.010 rad/s”인지에 따라 행이 별개.
- 0.01 단위 그라데이션만 잡아도 차원당 10^4 개, 4차원이면 10^{16} 개의 행 — 차원의 저주(curse of dimensionality)의 구체적 예.

로봇의 행동도 연속이다 — “관절에 정확히 +3.7Nm”은 이산 목록(열)에 없다. 연속값은 무한히 많아서 열 자체가 존재하지 않는다.

연속 → 두 가지 확장

상태가 연속이면 →

- 함수근사(신경망)로 Q 를 표현.
- Ch. 9, DQN

행동이 연속이면 →

- $\max_a Q(s, a)$ 계산 자체가 불가능.
- 정책 $\pi(a|s)$ 를 직접 학습.
- Ch. 10~11, REINFORCE / PPO

이 두 확장 — 상태를 신경망으로, 행동을 정책으로 — 가 바로 “딥강화학습”과 “정책기반 RL”.

왜 로봇인가

연속 상태 + 연속 행동 + 물리 법칙의 결합. 표 기반만으로는 풀 수 없고, 신경망 근사 + 정책기반 + 물리 엔진이 모두 필요한 문제.

시뮬레이션: 싼 현실

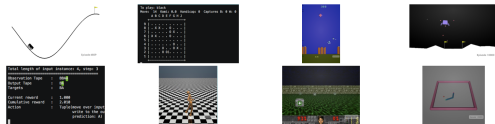
실제 로봇에서 10,000회 시행착오 = **느리고, 고장 날 위험, 안전 문제**. 표준적 해결책: **시뮬레이션**.

- **시뮬레이션 $\neq$ 현실** — 시뮬에서 학습한 정책이 실제 로봇에서 통하는가의 간극 = **sim-to-real gap** (13.1).
- 시뮬레이션은 “완벽한” 현실의 대체재가 아니라 “**싼**” 현실의 대체재.
- “로봇을 한 번도 구동하지 않고 **시뮬레이션만으로** 학습한 정책을 실제 로봇에 옮기는 것”이 최근 로봇 연구의 핵심 주제.

실습으로 연결 — CartPole(이산 2행동) vs Pendulum(연속 토크 $-2 \sim +2$): 환경 비교만으로 “이산 vs 연속” 차이가 보이고, **알고리즘 선택까지** 바뀐다. (1.3절에서 코드로 확인)

이번 학기의 실습 환경: Gymnasium

- **Gymnasium** — 옛 OpenAI Gym(Brockman et al., 2016)의 후신.
표준 인터페이스 라이브러리.
- 어떤 환경이든: 현재 상태를 관측(reset)하고, 행동을 하나 취하면(step) **다음 상태·보상·종료 여부**가 돌아온다.
- 인터페이스가 통일되므로 Ch. 2~11의 어떤 알고리즘이든 **환경만 바꿔 끼우면** 재사용.
- Ch. 13~14의 MuJoCo·Isaac Sim 로봇 환경도 이 reset/step 패턴을 그대로 따름.



Gym에 포함된 환경들 (CarRacing, Go, AirRaid, LunarLander, MountainCar, Walker2d, Doom, Reacher) — 원 논문 Figure 1.

reset/step으로 직접 보고

```
import gymnasium as gym

# (1) CartPole: 막대균형 . 차원4 연속상태 , 개2 이산행동
env = gym.make("CartPole-v1")
obs, _ = env.reset(seed=0)
print("CartPole 초기상태 :", obs)
print(" 상태공간 :", env.observation_space)
print(" 행동공간 :", env.action_space)
env.close()

# (2) Pendulum: 막대를위로균형 . 차원3 연속상태 , 연속** 행동
env = gym.make("Pendulum-v1")
obs, _ = env.reset(seed=0)
print("Pendulum 초기상태 :", obs)
print(" 상태공간 :", env.observation_space)
print(" 행동공간 :", env.action_space) # 연속: Box(-2, +2)
env.close()
```

지금은 env.action_space.sample()로 **완전 무작위**로 행동 — Ch.2~11을 거치며 이 한 줄이 “알고리즘이 고른 행동”으로 똑똑해지는 것을 본다.

이번 학기 로드맵

각 장의 목표와 핵심 산출물. “한 장의 A4”로 붙여놓고 “지금 여기 있다”를 확인하는 용도.

Block A (Ch. 2-8): “표 기반”의 완성

신경망 없이, RL의 이론적 뼈대를 표(table) 기반으로.

- 2 멀티암 밴딧: ϵ -greedy, UCB
- 3 MDP 정식화: 상태·전이·보상·할인율
- 4 동적계획법: 정책 반복, 값 반복
- 5 몬테카를로: 에피소드로 값 추정
- 6 시간차 학습: TD(0), Q-learning — 표 기반의 완성
- 7 n-step/적격흔적: 편향-분산 조절
- 8 Block A 캡스톤 + 중간고사

Block B (Ch. 9-16): 신경망 + 로봇

- 9 DQN: 신경망으로 Q 근사 (Atari 표준)
- 10 REINFORCE: $\max_a Q$ 없이 정책 직접 학습
- 11 PPO: 필수 알고리즘 (클래핑, KL 제어)
- 12 모방학습 & RLHF: 시연·선호로부터
- 13 로봇 시뮬레이션: Sim-to-Real, 보상 설계
- 14 MuJoCo & Isaac Sim: GPU 가속 물리
- 15 모델기반 RL & MCTS (AlphaGo 핵심)
- 16 Block B 캡스톤 + 학기 총정리

Ch. 2~7의 순서는 Sutton-Barto 교과서 목차 그대로 — 더 단순한 문제에서, 한 단계씩 도구를 추가한다.

로드맵을 한 장의 흐름도로

Block A는 “표 기반” 도구들을 **순서대로 쌓고**, Block B는 그 도구를 **신경망과 로봇**으로 확장.



Block A(Ch. 2~7): 밴딧 → MDP → DP → MC → TD → n-step/적격훈적. Block B(Ch. 9~15): DQN·정책기반 → 모방학습/RLHF → 로봇 시뮬레이션(MuJoCo·Isaac Sim) → 모델기반 RL/MCTS. 중간에 캡스톤(Ch. 8, 16).

“표 기반의 완성”이 왜 중요한가

Block A를 “지루한 기초”로 넘기면 Block B에서 왜 그 기법이 그렇게 생겼는지 이해 못 한다.
신경망 RL은 표 기반 RL의 “더 큰 버전”이지, 별도 영역이 아니다.

- DQN의 **경험 재사용(experience replay)** = MC의 “에피소드를 모아서 평균”의 신경망 버전.
- DQN의 **타깃 네트워크** = “자기 예측을 정답으로 삼으면 불안정해 진다”는 TD의 근본 문제(Ch. 6)를 신경망 버전에서 해결.
- PPO의 **클래핑(clipping)** = “한 번에 정책을 너무 많이 바꾸면 발산”하는 실무 버전.

블록 B의 모든 기법은 기존 도구의 **자연스러운 확장**으로 읽힌다.

ML1과의 연결: 이 과목은 ML1의 “다음”

한 문장

ML1은 “주어진 데이터를 어떻게 모델로 변환하는가”를, ML2는 “행동하며 데이터를 스스로 만들어내는 에이전트를 어떻게 학습시키는가”를 다룬다.

ML1 개념	ML2에서
“ y 가 있나? 보상이 있나?” (1.2) 신경망 + 역전파 (Ch. 9)	이 과목의 입구 — “보상이 있다”는 갈래가 곧 RL의 정의 (Ch. 2~) DQN의 Q-네트워크, REINFORCE/PPO의 정책 네트워크 — 그대로 사용
손실함수 + 경사하강법 (Ch. 2, 9) 과적합·정규화 (Ch. 6) LLM 정렬 (Ch. 13)	DQN의 TD 손실, PPO의 정책 손실 — 같은 프레임워크 DQN 경험재사용·타깃네트워크, PPO 클래핑이 이 역할 RLHF (Ch. 12) — 같은 기술의 “사람 선호를 보상으로” 쪽에서 재회
“ML1에서 한 줄로 지나간 개념이 ML2에서 한 장이 된다” — 이 감각이 ML1→ML2를 하나의 연속 학문으로 만든다.	

확인 문제 + 자주 묻는 질문

확인 문제: “보상”이 “정답”으로 바뀌면?

- (a) 성공/실패 **라벨이 붙은** 영상 10,000개로 패턴 학습 → **지도학습**(시연 데이터) = **모방학습** (Ch. 12).
- (b) 시연 **100개뿐**이고 나머지는 스스로 행동하며 만들면 → **강화학습**, 희박 보상 “성공 시 +1”, 탐험 vs 활용 딜레마가 여기서 등장.
- 경계: **시연이 충분하면 모방, 부족하면 강화.**

자주 묻는 질문

- “RL은 지도학습의 특수한 경우?” → 아니다. **데이터를 누가 만드냐**가 본질적 차이.
- “보상이 희박하면 불가능?” → 불가능하진 않으나 훨씬 어려움. 도구: TD(Ch. 6) · 적격흔적(Ch. 7) · MCTS(Ch. 15) · **보상 성형**(Ch. 13~14).
- “실제 로봇을 다루냐?” → 아니다, **시뮬레이션**. 시뮬만으로도 충분한 이유: 실제 로봇 학습은 위험·비쌈·느리고, sim-to-real이 표준 방법이 되었기 때문.
- “모델 안다 vs 모른다?” → ML1의 생성적 vs 판별적 접근과 같은 “중간 모델을 세울 것인가” 질문. RL에서는 **모델기반**(Ch. 15) vs **모델프리**(Ch. 2~11).
- “왜 PPO가 필수?” → REINFORCE는 실무에 불안정(한 번에 너무 많이 바꾸면 발산). PPO는 **클래핑 + KL 제어**로 잡아준 실무 표준.

1.2 신경망 미니 리뷰

“열쇠 정리” 시간

강화학습을 **새로** 가르치는 게 아니다 — 앞으로 16개 챕터 내내 쓸 **신경망 뼈대**를 한 번 더 정확히 짚는 시간.

이번 학기에서 신경망은 **정책(policy)**이 된다: 상태 s 를 입력받아 행동 a 를 내보내는 $\pi_{\theta}(a|s)$. 이 함수가 어떻게 동작하고 학습되는지 모르면 Ch. 9(DQN)부터 Ch. 11(PPO)의 모든 수식이 “마법”으로만 보인다.

50분 안에 남기는 네 가지

(1) 순전파가 정확히 뭘 하는지, (2) 층을 쌓으면 표현력이 기하급수적으로 폭발하는 이유, (3) 역전파가 숫자 수준에서 실제로 무엇인지, (4) 이 모든 것이 RL에서 어떤 역할을 하는지.

순전파: 4개의 식이 전부

$$z_1 = W_1 x + b_1, \quad a_1 = \sigma(z_1), \quad z_2 = W_2^T a_1 + b_2, \quad a_2 = \sigma(z_2)$$

- W_1 — 1층(은닉층) 가중치: 각 유닛이 입력의 어떤 조합을 “보느냐”.
- b_1 — **활성 임계값**을 이동시키는 편향.
- z_1 — 활성화 전의 “선형 신호”.
- σ — **활성 함수**: a_1 이 이 신호를 **비선형**으로 왜곡해 내보낸다.
- $W_2^T a_1 + b_2$ — 은닉 신호를 가중 합쳐 최종 예측.

이 4개의 식이 신경망의 **전부**다 — 나머지(프레임워크·GPU·대규모 데이터)는 이 4줄을 빠르고 안정적으로 계산하는 기술에 불과.

학습의 정의

학습(training)

예측이 얼마나 틀렸는지를 **손실함수** J 로 수치화하고, J 를 **줄이는 방향**(그래디언트의 반대 방향)으로 파라미터 W_1, W_2, b_1, b_2 를 조금씩 옮기는 과정.

이번 학기에서는 PyTorch의 `nn.Module`과 `.backward()`로 이 계산을 **자동**으로 처리 — 손으로 유도하는 연습은 (ML1을 들었다면) 끝냈다고 가정.

그럼에도 “왜 활성화 함수 σ 가 있는가?” 이 절에서 가장 중요한 “왜”.

활성 함수가 없으면: 두 층이 하나의 선형 변환으로

σ 를 빼고 (활성 함수 없는) 2층만 쓰면, 두 층은 **하나의 선형 변환으로 완전히 축소된다**:

$$a_2 = W_2^T (W_1 x + b_1) + b_2 = (W_2^T W_1) x + (W_2^T b_1 + b_2) \equiv W x + b$$

- 두 층이 하나의 층으로 **합쳐져 버린다**.
- 아무리 층을 쌓아도 **직선(회귀)** 또는 **평행사변형의 경계(분류)**밖에 그릴 수 없다.
- 활성 함수가 없으면 층의 “깊이”라는 개념 자체가 존재하지 **않는다**.

이 등식($W_2^T W_1 \equiv W$)을 코드로 직접 확인하는 것은 1.2 실습 노트북.

비선형 활성이 있는 층을 쌓으면

- 은닉 유닛 하나가 “ x 가 이 방향에 있는가”를 확인하는 한 개의 **선형 경계**(hyperplane)를 만든다.
- 16개 유닛 → **16개의 서로 다른 경계**.
- 경계들이 x 공간을 **여러 조각**(각각 다른 선형 영역)으로 쪼개고, 마지막 선형 층이 각 조각 안에서 **각각 다른 기울기**의 직선을 이어붙인다.

깊이가 곧 표현력의 지수

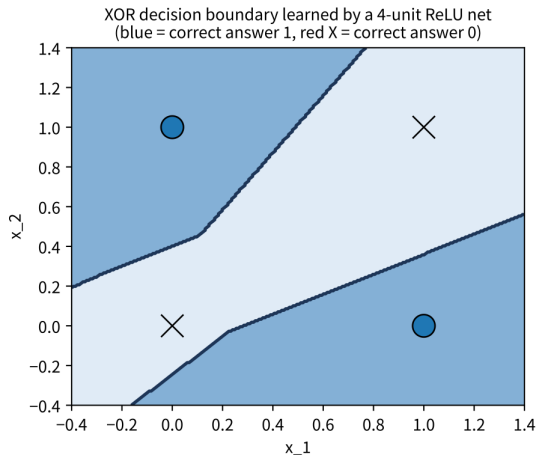
층이 하나 더 쌓일 때마다 이 “조각 쪼개기”가 **기하급수적으로** 세진다.

XOR: 직선으로는 안 된다

두 입력 $x_1, x_2 \in \{0, 1\}$, 둘이 다를 때만 $y = 1$:

x_1	x_2	정답 y
0	0	0
0	1	1
1	0	1
1	1	0

- 정답 1인 점 $(0, 1), (1, 0)$ 은 “한 대각선”, 0인 점 $(0, 0), (1, 1)$ 은 “다른 대각선”.
- 한 개의 직선으로 분리하는 경로 존재하지 않음.
- XOR은 **선형 분리 불가능**.



4-유닛 ReLU 신경망을 역전파로 학습해 얻은 XOR 결정

경계 — 여러 선형 경계가 만나 한 직선으로는 불가능했던

분리. (파란 점 = 1, 붉은 × = 0)

2층 신경망으로 XOR 풀기 + 역사

- 은닉 각 유닛이 $z = \max(0, w_1x_1 + w_2x_2 + b)$ — **한 개의 직선 경계**.
- 방향이 다른 두 유닛(“ x_1+x_2 큰 쪽”과 “ x_1-x_2 큰 쪽”)의 두 직선이 2차원을 **4개 영역**으로 쪼갬다.
- 출력층이 각 영역에 다른 가중치를 두어 **파란 두 점에만 높은 값**. (실제 학습엔 **4개 이상**의 유닛 필요 — 2유닛은 잘 알려진 로컬 미니멈에 빠지기 쉽다.)

역사적 맥락

XOR은 1969년 민스키·파피트의 저서 Perceptrons에서 “단층 퍼셉트론으로 풀 수 없는 가장 간단한 문제”로 지적되며 **신경망 연구 10년 정지**의 결정적 원인. 1986년

역전파(Rumelhart-Hinton-Williams)로 해소 — 2층 이상은 XOR을 풀고, 그 그래디언트를 계산하는 방법이 있다.

왜 ReLU인가(기본 활성화 함수)

시그모이드 $\sigma(z) = 1/(1 + e^{-z})$

- 비선형이고 XOR은 풀 수 있다.
- z 가 아주 크거나 작으면 도함수 $\approx 0 \rightarrow$ 층이 깊어질수록 그래디언트가 지수급으로 작아지는 **사라지는 그래디언트**(ML1 Ch. 9).

ReLU $\max(0, z)$

- $z > 0$ 에서 도함수가 **정확히 1** — 사라지는 그래디언트가 훨씬 완화.
- 이번 학기 실습의 기본 활성화 함수.

단면 예: 1-2-1 순계산(다음)에서 $z = -1$ 인 유닛은 ReLU 도함수 0으로 “죽어” 이번 업데이트에서 전혀 움직이지 않는다 — **희소 활성화(sparse activation)**.

역전파는 왜 필요한가

그래디언트를 구하는 방법 = **역전파(backpropagation)** — 연쇄법칙을 출력에서 입력 방향으로 거꾸로 적용해, 각 파라미터가 최종 손실에 얼마나 책임이 있는지 계산.

- 층이 하나면 미분을 손으로 바로 계산 가능.
- 층을 여러 개 쌓으면 한 파라미터의 변화가 최종 손실에 도달하기까지 **경로가 길어진다**.
- 역전파 = 이 긴 경로를 “출력의 오차를 **한 층씩 거슬러 올려보내며**” 정확하고 효율적으로 계산하는 절차.

그래도 한 번은 숫자로. “오차를 거슬러 올려보낸다”가 실제로 무슨 뜻인지, 1-2-1 신경망(입력 1, 은닉 2, 출력 1, ReLU)으로 손계산. 가중치를 단순하게 골라 **정수 연산**만으로 끝낸다.

1-2-1 손계산: 설정과 순전파

- 은닉층: $W_1 = \begin{bmatrix} +2 \\ -1 \end{bmatrix}, b_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

출력층: $W_2 = \begin{bmatrix} 1 & 1 \end{bmatrix}, b_2 = 0$

- 입력/정답: $x = 1, y = 3$

손실: $J = (\text{pred} - y)^2$

순전파:

$$z_1 = \begin{bmatrix} 2 \cdot 1 \\ -1 \cdot 1 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad a_1 = \text{ReLU}(z_1) = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

$$\text{pred} = W_2 a_1 + b_2 = 1 \cdot 2 + 1 \cdot 0 = 2, \quad J = (2 - 3)^2 = 1, \quad \text{err} = \text{pred} - y = -1$$

두 번째 은닉 유닛이 **죽었다**($\text{ReLU}(-1) = 0$) — 이 유닛의 그래디언트가 **정확히 0**이 되는 것을 곧 본다.

1-2-1 손계산: 역전파 (출력층)

출력층 그래디언트(연쇄법칙, 가장 단순한 층부터):

$$\frac{\partial J}{\partial W_2} = 2 \text{err} \cdot a_1^T = 2(-1) \begin{bmatrix} 2 & 0 \end{bmatrix} = \begin{bmatrix} -4 & 0 \end{bmatrix}, \quad \frac{\partial J}{\partial b_2} = 2 \text{err} = -2$$

은닉층 그래디언트 — 출력층의 “오차 신호” $2 \text{err} \cdot W_2$ 를 내려보내고 ReLU 도함수로 걸러냄:

$$\frac{\partial J}{\partial a_1} = 2 \text{err} \cdot W_2^T = 2(-1) \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ -2 \end{bmatrix}$$

$$\frac{\partial J}{\partial z_1} = \frac{\partial J}{\partial a_1} \odot \text{ReLU}'(z_1) = \begin{bmatrix} -2 \\ -2 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -2 \\ 0 \end{bmatrix}$$

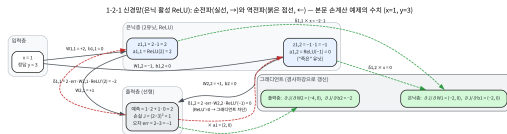
1-2-1 손계산: 역전파 (끝) + 흐름도

$$\frac{\partial J}{\partial W_1} = \frac{\partial J}{\partial z_1} x^T = \begin{bmatrix} -2 \\ 0 \end{bmatrix} \cdot 1 = \begin{bmatrix} -2 \\ 0 \end{bmatrix}, \quad \frac{\partial J}{\partial b_1} = \frac{\partial J}{\partial z_1} = \begin{bmatrix} -2 \\ 0 \end{bmatrix}$$

- 두 번째 유닛의 그래디언트가 **정확히 0** — “아무 책임도 없다”. 죽은 유닛은 이번 업데이트에서 **전혀 움직이지 않는다** (희소 활성화).

- 이 네 줄(W_2, b_2, W_1, b_1)이 **역전파의 전부**. PyTorch의 `.backward()`가 내부에서 자동으로 하는 것이 **정확히 이것** — 층이 100개여도 같은 4줄(각 층마다 반복).

- 각 단계를 코드로 재현하면 `.backward()`가 채운 `model.net[i].weight.grad`가 손계산과 **정확히 일치** — “역전파가 내가 생각한 것과 같다”는 신뢰의 근거.



1-2-1 역전파 흐름도 — 순전파(실선)와 역전파(붉은 점선)가 위 수치와 대응. 죽은 유닛은 그래디언트 0.

연쇄법칙의 구조를 한 줄로

$$\frac{\partial J}{\partial W_1} = \left(\frac{\partial J}{\partial a_1} \right) \odot \text{ReLU}'(z_1) \cdot x^T$$

세 인자의 곱

“출력층에서 내려온 **오차**” × “이 유닛의 **활성 도함수**” × **입력** — 이 세 인자의 곱이 **모든 신경망의 모든 층의** 그래디언트를 만든다.

층이 깊어질수록 이 곱이 계속 **쌓이고** — 이것이 역전파가 “오차를 한 층씩 거슬러 올려보낸다”는 말의 정확함이다.

유한차분으로 그래디언트를 확인: 단위 테스트

단일 가중치의 가장 작은 예제: $\text{pred} = w x_1 + w_2 x_2 + b$, $w = 0.3$ (배울 파라미터), $w_2 = -0.7$, $b = 0.2$ 고정. 데이터 $x_1 = 0.4$, $x_2 = 0.9$, $y = 1.0$, $J = (\text{pred} - y)^2$.

계산

손계산 $\text{pred} = 0.3(0.4) + (-0.7)(0.9) + 0.2 = -0.31$; $J = (-1.31)^2 = 1.7161$

$$\frac{\partial J}{\partial w} = 2(\text{pred} - y) x_1 = 2(-1.31)(0.4) = -\mathbf{1.048}$$

유한차분 w 를 ± 0.001 : $J(0.301) = 1.715052$, $J(0.299) = 1.717148$

중심차분: $(J(0.301) - J(0.299))/0.002 = -\mathbf{1.048}$ — 정확히 일치

해석적 미분 vs 유한차분의 일치 확인이 “역전파가 맞는가?”의 가장 간단한 **단위 테스트** = ML1에서 배운 **그래디언트 체크**. 역전파 코드를 직접 쓰는 순간(Ch. 10 정책 그래디언트), “내 미분이 맞나?”에 대한 **유일한 객관적 검증**.

실습: PyTorch로 $y = 2x + 1$ 배우기

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, in_dim, hidden_dim, out_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, out_dim),
        )

    def forward(self, x):
        return self.net(x)

torch.manual_seed(0)
model = MLP(1, 16, 1)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.MSELoss()

X = torch.rand(64, 1) * 4 - 2
y = 2 * X + 1
```

```
for epoch in range(200):
```

세 줄 루프: `zero_grad()` → `backward()` → `step()`

- ① `optimizer.zero_grad()` — 이전 에포크의 그래디언트를 **초기화**. PyTorch는 `.backward()` 반복 시 그래디언트가 **축적(accumulate)** → 매 업데이트 전에 **반드시** 0으로 비워야. 빠뜨리면 그래디언트가 계속 쌓여 **학습이 발산**.
- ② `loss.backward()` — **역전파** 실행. 손계산한 4줄의 그래디언트를 모든 파라미터에 자동 계산해 `.grad`에 채움.
- ③ `optimizer.step()` — $\theta \leftarrow \theta - \eta \nabla J$ 실행. Adam은 모멘텀 + 적응적 학습률로 보정.

이번 학기 내내 반복되는 리듬

이 세 줄이 Ch. 9(DQN), Ch. 10~11(정책 신경망)에서 **똑같이** 반복된다.

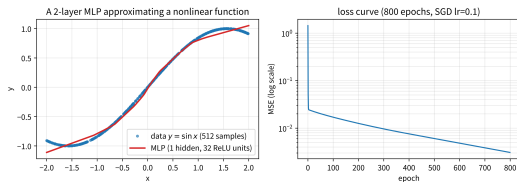
비선형성이 실제로 필요한 순간: $y = \sin(x)$

$y = 2x+1$ 은 직선 — 선형 모델 하나로도 풀린다.

비선형 $y = \sin(x)$ 를 같은 2층 MLP로 근사:
(numpy MLP, 은닉 32유닛, ReLU, SGD lr=0.1,
 $x \in [-2, 2]$, 512샘플, 800 에포크)

- 32개의 직선 조각이 이어붙여 곡선이 된다
— $\sin(x)$ 곡률을 따라 “계단식 근사”.
- 손실: $1.44 \rightarrow 0.017 \rightarrow 0.008 \rightarrow 0.003$
(1/100/400/800 에포크) — **지수적 감소**,
신경망 학습의 전형적 패턴.
- 직선으로 불가능했던 근사가 비선형
2층으로 가능 — “비선형성의 힘”.

주의: 32유닛 계단식 근사는 극값 ($x = \pm\pi/2$)과 양
끝($x = \pm 2$)에서 오차가 가장 크다 — 곡률이 센 곳. 유닛을
늘리면 매끄러워진다.



왼쪽: 800에포크 학습 후 근사(파란 원 = 데이터, 붉은
곡선 = MLP). 오른쪽: 손실 곡선(MSE, 로그 스케일),
최종 ≈ 0.0031 .

신경망 학습의 세 가지 “왜”를 정리

질문	답
왜 비선형 활성화가 필요한가?	없으면 여러 층이 하나의 선형 변환으로 축소 → 깊이 무의미
왜 2층 이상인가?	1층 = 선형 분리만 → XOR 불가. 2층 = 선형 경계의 조합 → 임의의 결정 경계 근사
왜 역전파인가?	층이 깊을수록 손으로 미분 불가 → 연쇄법칙을 출력에서 입력으로 자동 적용
왜 ReLU인가(기본)?	시그모이드의 사라지는 그래디언트 완화, $z > 0$ 에서 도함수 = 1으로 계산 저렴
왜 zero_grad()를 매번?	PyTorch 그래디언트 추적 → 빠뜨리면 발산

이 뼈대가 강화학습에서 어떻게 쓰이는가

Q-학습의 신경망 버전 (DQN, Ch. 9)

- **모델:** $Q_{\theta}(s, a)$ — 상태-행동 쌍의 “장래 누적 보상 예측”
- **손실:** $J = \mathbb{E} [(r + \gamma \max_{a'} Q_{\theta-}(s', a') - Q_{\theta}(s, a))^2]$
— 벨만 방정식(Ch. 4)의 “왼쪽”과 “오른쪽”의 오차
- **학습:** 동일한 세 줄

정책 기반 (REINFORCE, PPO, Ch. 10~11)

- **모델:** $\pi_{\theta}(a|s)$ — 각 행동의 확률 (출력층 소프트맥스)
- **손실:** $-\sum_t \log \pi_{\theta}(a_t|s_t) \cdot G_t$ — “보상이 큰 행동의 확률을 올린다” (정책 그래디언트 정리)
- **학습:** 여전히 동일한 세 줄

핵심 인사이트: 새로움은 손실함수의 정의에

지도학습: $x \rightarrow \hat{y} \rightarrow J = (\hat{y} - y)^2 \rightarrow$ 역전파 $\rightarrow W$ 업데이트. 정답 y 가 미리 주어진다.

강화학습: $s \rightarrow \pi_{\theta}(s) \rightarrow$ 보상 r 으로 평가 $\rightarrow$ 정책의 가치를 낮추는 방향으로 W 업데이트. 정답 y 가 없다 — “이 행동이 얼마나 좋은가”를 보상으로 간접 평가.

	지도학습(ML1)	DQN(Ch. 9)	REINFORCE/PPO(Ch. 11)
모델 출력	$\hat{y}$ (라벨 예측)	$Q(s, a)$ (가치 예측)	$\pi(a s)$ (행동 확률)
손실 정의	$(\hat{y} - y)^2$ 또는 교차엔트로피	벨만 오차의 MSE	정책 로그-가능성 $\times$ 보상
“정답” 출처	데이터셋 라벨	보상 + 다음 상태 Q값	보상(누적)
학습 루프	3줄 동일	3줄 동일	3줄 동일

이번 학기를 통과하는 가장 중요한 열쇠

RL의 “새로움”은 학습 알고리즘이 아니라 손실함수의 정의. 기계(`zero_grad()` $\rightarrow$ `backward()` $\rightarrow$ `step()`)는 그대로고, 그 기계에 무엇을 “정답”으로 넣을 것인가가 지도/강화를 가른다.

자주 하는 실수 (1): 초기화와 zero_grad()

- **실수 1: “층을 더 쌓으면 무엇이든 잘 배운다”** — 아니다. 초기화가 잘못되면(가중치가 너무 크거나 작으면) 신호가 층을 거치며 **폭주하거나 사라져** 학습이 시작조차 안 됨. MLP(1, 16, 1)의 16이 “적당히” 큰 이유: PyTorch nn.Linear의 **Kaiming 초기화**(입력 차원에 반비례 균등 분포, He et al., 2015). 1024로 늘리면 느려지거나 발산. **깊이는 표현력을 주지만, 안정성은 초기화가 보장.**
- **실수 2: zero_grad()를 빼먹기** — .backward() 반복 시 그래디언트 **축적**. 빠뜨리면 “에포크 1 + 에포크 2의 그래디언트”로 업데이트 → 발산/비수렴 “이상한 현상”의 **가장 흔한 원인**. 매 업데이트 전에 **zero_grad()**.

자주 하는 실수 (2): NaN과 “자동화니까 원리 몰라도 된다”

- **실수 3: 손실이 NaN인 것 무시하기** — 한 번 nan이 되면 모든 예측이 nan이고 학습은 **영원히 멈춤**. 원인: (a) 학습률이 너무 커서 업데이트가 손실함수의 오목한 영역을 벗어나 **발산**, (b) $\log(0)$, $0/0$ 같은 **수치적 특이점**. $y=2x+1$ 예제에서 $lr=1.0$ 으로 바꾸면 50에포크도 안 돼 nan. 직전 에포크의 손실이 “갑자기 튀었는가”부터 보라 — 대개 학습률 문제.
- **실수 4: “역전파가 자동으로 되니까 원리를 몰라도 된다”** — “마법”인 줄 알면 “왜 학습이 안 되지?”를 debug할 수 없다. Ch. 10에서 정책 그래디언트를 직접 유도할 때, Ch. 12에서 “왜 이 손실을 쓰나?”를 물을 때 역전파의 구조가 직접 필요하다. **디버깅과 설계를 하려면 원리가 필요하다.**

1.3 실습 환경 세팅: Gymnasium

왜 표준 인터페이스가 중요한가

이번 학기 실습은 **Gymnasium**(Towers et al., 2024) — 옛 OpenAI Gym의 후신 — 위에서 진행한다.

- 어떤 환경이든 “현재 상태를 관측(reset), 행동을 하나 취하면(step) **다음 상태·보상·종료 여부가 돌아온다**”는 **같은 인터페이스**.
- 2016년 Gym(Brockman et al., 2016) 이전: 각 연구자가 각자 시뮬레이터를 각자 감쌌고, 재현하려면 “이 사람의 환경 API를 읽는 것”부터.
- Gym은 “**모든 환경이 동일한 두 함수 (reset/step)를 제공해야 한다**”는 규칙 하나로 수천 개의 환경을 하나의 코드베이스 위에서 비교 가능하게.
- 이름 그대로 gym(체육관) — “에이전트를 훈련시키러 들어가는 곳”. 이후 **Gymnasium 포크**가 사실상 표준. 이번 책의 코드는 **Gymnasium 1.x** 기준.

표준 계약: reset과 step

```
obs, info = env.reset(seed=42)
# ... 정책을 obs 보고행동을 a 정한다 ...
obs_next, reward, terminated, truncated, info = env.step(a)
```

- reset — 에피소드(한 번의 게임·시도) 시작, **초기 상태** obs를 돌려준다.
- step — 한 행동을 받아 **다섯 개**를 돌려준다: 다음 관측 obs_next, 한 스텝의 **보상** reward, 종료 신호 두 가지 terminated/ truncated(아래에서 구분), 부가 정보 info.

이 계약이 1.1절의 강화학습 정의를 코드로 옮긴 것:

$$s_{t+1} = f(s_t, a_t), \quad \text{보상 } r_{t+1} \text{ 을 함께 받는다}$$

f = 환경의 **전이(transition)**. f 를 **정확히 안다면** 모델 기반, **모르면** 모델 프리 — Ch. 5~7이 다루는 것은 후자.

환경 만들기: gym.make

```
import gymnasium as gym

env = gym.make("CartPole-v1")    # 막대를 쓰러뜨리지 않고 균형 잡기
obs, info = env.reset(seed=0)    # 에피소드 시작 -> 초기 관측
print(type(env), " | 이름:", env.spec.id)
env.close()
```

- 환경은 **이름 문자열 하나로** — EnvSpec에 등록돼 있는 대로 인스턴스화.
- -v1, -v2 서명: 같은 게임이라도 물리 파라미터·종료 조건이 다른 **버전** 구분.
- make와 reset **분리**: make는 환경 객체만 만들고 실제 시뮬레이션은 reset이 **시작**.
같은 환경 객체를 **여러 에피소드·여러 시드**에 걸쳐 다시 쓰는 것이 이 구조의 핵심.

CartPole이란 무엇인가

강화학습의 “hello world”.

- 수평 레일 위를 미는 **카트** + 그 위에 **균형**으로 서야 하는 **막대(pole)**.
- 카트를 **좌(0)**·**우(1)**로만 미는 것이 허용.
- 목표: 막대가 **12도(± 0.2094 rad)** 이상 기울어지지 않게 최대한 오래 버티기.
- 물리적으로는 **역행자 매달린 진자(inverted pendulum)** — 평형이 **불안정**. 아무것도 안 하면 필연적으로 쓰러진다.
- 조금 기울어지는 순간 관성이 기울어짐을 **증폭** → “한 번 밀어주고 끝”이 아니라 **매 스텝 대응** 필요. ⇒ **무작위 에이전트는 못 푼다** — 다음 장의 탐험/활용 논의로 자연스럽게 연결.

종료 조건 세 가지, 종료 플래그는 두 종류

조건	내용	플래그
1	막대 각도 $ \theta $ 가 0.2094 rad(약 12도) 초과 → 진짜 실패	terminated
2	카트가 레일 끝($ x > 2.4$ m) 벗어남 → 환경이 진짜 실패 로 처리	terminated
3	500 스텝 채움(막대는 아직 서 있다) → 시간 한도	truncated

즉 조건 1·2는 모두 **terminated**, truncated는 **시간 한도(조건 3)**뿐이 세운다.

보상: 살아 있는 한 스텝마다 $+1$ — “버틴 스텝 수”가 곧 에피소드 리턴. 막대각도·중앙에서 멀어짐 같은 **중간 신호를 주지 않고** “아직 살아 있는가”에만 상 → 에이전트가 “어떻게 버틸 것인가”를 스스로 발견해야 — 교육용으로 좋은 이유. (“각도 0에 가까울수록 $+\alpha$ ” 같은 **보상 성형**은 보상이 정말 희박할 때(Ch. 13~14 로봇)의 별도 주제.)

관찰 공간과 행동 공간: Box와 Discrete

```
env = gym.make("CartPole-v1")
obs, _ = env.reset(seed=0)
print("관측 공간 :", env.observation_space)
print("행동 공간 :", env.action_space)
print("초기 관측 :", obs)
```

관측 공간 : Box([-4.8 -inf -0.41887903 -inf], [4.8 inf 0.41887903 inf], (4,), float32)

행동 공간 : Discrete(2) 초기 관측 : [0.0137 -0.023 -0.0459 -0.0483]

- **관측은 4차원** $[x, \dot{x}, \theta, \dot{\theta}]$ (카트 위치·속도, 막대 각도·각속도). Box로 각 성분의 상·하한(low/high)을 표현. $x \in [-4.8, 4.8]$, $\theta \in [-0.419, 0.419]$, 속도·각속도는 $\pm\infty$.
- **행동은 Discrete(2) — 이산 두 개** (0: 왼쪽, 1: 오른쪽). `action_space.n == 2`로 확인.

왜 관측이 4개인가: 위치 + 속도 = 상태

- 위치만 주면(막대가 지금 5도 기울어 있다고만), “아직 기울어지고 있는 중인가, 기울어짐이 멈췄는가”를 알 수 없어 다음 행동을 결정 못 한다.
- 속도까지 주어야 상태가 **완전히 기술**된다.
- “위치 + 속도” 짝이 물리 시스템 관측의 기본 단위. 이 쌍이 **상태(state)**를 이룬다는 사실은 Ch. 4에서 MDP의 s_t 를 정의할 때 다시 만난다.

CartPole — Discrete(2): 행동이 **이산**으로 2개. **Pendulum** — Box(-2.0, 2.0, (1,), float32): “얼마의 토크(-2 ~ +2)를 줄지”를 실수 하나로 — **연속**.

이산/연속이 행동 공간의 **형태**를 정하며, 이 차이가 이후 **어떤 Q함수.정책을 표현할지**를 결정 (Ch. 9~11: DQN은 이산 행동을, 정책기반은 연속 행동을 자연스럽게).

실습: CartPole에서 무작위 정책 실행

```
import gymnasium as gym

env = gym.make("CartPole-v1") # 막대를 쓰러뜨리지 않고 균형 잡기
obs, info = env.reset(seed=0)
print("초기 상태카트( 위치, 속도, 막대각도 , 각속도):", obs)

for _ in range(3):
    action = env.action_space.sample() # 지금은 무작위 정책
    obs, reward, terminated, truncated, info = env.step(action)
    print("행동:", action, "-> 다음상태 :", obs, "보상:", reward)

env.close()
```

지금은 `env.action_space.sample()`로 **완전 무작위**로 행동 — Ch.2~11을 거치며 이 한 줄이 “알고리즘이 고른 행동”으로 **점점 뚝뚝해**지는 것을 본다. 이 CartPole은 표 기반 알고리즘을 **시각적으로** 이해하는 데, 그리고 Ch.9의 DQN에서 **다시** 등장한다.

에피소드가 끝나면: terminated와 truncated를 구분

둘 다 “이 에피소드는 여기까지”지만 끝난 이유가 다르다.

terminated = True

- 환경 자체가 “게임이 끝났다”고 선언.
- CartPole: 막대 12도 초과, 카트 레일 이탈.
- 터미널 상태 — 그 뒤의 미래 가치 $V(s_{t+1}) = 0$ (게임이 끝났으니까).

truncated = True

- 환경이 끝나지 않았으나 사람이 정한 한도(최대 스텝 수) 도달로 강제 종료. CartPole: 500 스텝.
- 게임은 계속됐을 것이다 — 다음 상태의 가치는 그대로 $V(s_{t+1})$ 을 써야.

왜 나누는가

가치함수를 재귀적으로 업데이트할 때(Ch. 5~7 MC/TD) 이 둘이 다르다. 에피소드 종료 판단은 `done = terminated or truncated`로 하되, 가치 업데이트에서는 terminated만 “미래 가치 0” 신호로 쓰는 것이 정석. 구분하지 않으면 500스텝째에 Q값을 0으로 잘라 학습이 부정확해진다.

시드(seed)와 재현성

RL은 **확률적**이다 — 초기 상태의 무작위성, 행동 샘플링의 무작위성, 전이의 확률성. “같은 코드를 돌렸는데 결과가 다르다”는 **버그가 아니라 정상**.

```
import numpy as np
env = gym.make("CartPole-v1")
obs1, _ = env.reset(seed=42)
obs2, _ = env.reset(seed=42)
print("같은 시드 -> 같은초기관측 :", np.allclose(obs1, obs2)) # True

env.action_space.seed(7); a1 = env.action_space.sample()
env.action_space.seed(7); a2 = env.action_space.sample()
print("같은 시드 -> 같은행동 :", a1 == a2) # True
```

- **다른 시드** → **다른 무작위 설정** — “여러 시드로 평가해서 평균 내자”의 실전 관행의 이유.
- `action_space.sample()`의 시드와 `reset`의 시드는 **독립된 무작위 스트림** — 초기 상태를 고정해도 **행동은 고정되지 않는다**(반대도). 둘 다 고정해야 완전히 결정적 재현.

자주 하는 실수 4가지

- ❶ **gym.make에서 바로 step을 부른다** — make는 환경만 만들고 초기화는 안 함. 반드시 reset이 먼저. reset 없이 step이면 ResetNeeded: Cannot call env.step() before calling env.reset() 오류.
- ❷ **done을 하나로만 취급한다** — 종료 판단에 terminated or truncated는 맞되, **가치 업데이트** 시 둘을 구분하지 않으면 500스텝째(터미널이 아닌)를 0으로 잘라 학습이 부정확.
- ❸ **보상을 누적하지 않는다** — step은 **한 스텝**의 보상만 줌. 에피소드 리턴은 `total += reward`로 매 스텝 누적해야.
- ❹ **env.close()를 깜빡한다** — 작은 환경에선 대수롭지 않으나 시뮬레이터 환경(Ch. 13~14)은 메모리를 계속 점유 → try/finally나 with 블록으로 정리하는 습관.

확인 문제

- 1 `env.observation_space.shape`와 `env.action_space.n`을 출력하면 CartPole의 관측 차원과 행동 개수는 몇인가? (힌트: 관측은 **위치+속도 4개**, 행동은 **이산 2개**.)
- 2 `terminated`와 `truncated`는 항상 **배타적**(둘 중 하나만 참)인가? (힌트: CartPole 자체는 한 스텝에 하나의 플래그만 세운다. 그런데 막대가 **500번째 스텝에서** 쓰러진다면? `gym.make`가 자동으로 덧붙이는 **TimeLimit 래퍼**가 `truncated`를 어떻게 세우는지를 생각하라.)
- 3 CartPole 보상이 “매 스텝 +1”인 것은 **보상 성형** 관점에서 어떤 성질인가? (힌트: 성형이 없는 보상 vs 있는 보상의 차이 — 1.1절의 “보상 = 얼마나 좋았는지의 점수”와 연결.)

무작위 정책의 “베이스라인”을 수치로 잡다

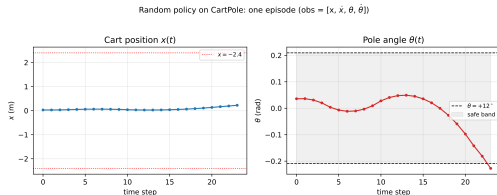
“무작위로 행동하면 얼마나 버티는가”를 **숫자로** 알아두면, 이후 “배운 정책”의 성적을 대조할 **기준점**이 생긴다. 200개 시드의 무작위 에피소드:

```
import numpy as np
env = gym.make("CartPole-v1")
lengths = []
for seed in range(200):
    s, _ = env.reset(seed=seed)
    env.action_space.seed(seed)    # 행동무작위성도고정    -> 완전히재현가능
    total, n = 0.0, 0
    while True:
        a = env.action_space.sample()
        s, r, term, trunc, _ = env.step(a)
        total += r; n += 1
        if term or trunc: break
    lengths.append(n)
lengths = np.array(lengths)
print(f"무작위 정책 (200 에피소드): 평균 {lengths.mean():.1f} 스텝, "
      f"최소 {lengths.min()}, 최대 {lengths.max()}")
```


결과: 평균 24.1 스텝

무작위 정책 (200 에피소드): 평균 24.1 스텝, 최소 9, 최대 100

- 무작위여도 평균 **24스텝**은 버틴다 — 초기 $|\theta| \approx 0.05$ rad가 작고, 좌/우를 반반씩 미니까 **운 좋게** 짧은 구간을 버팀.
- 500스텝 채우기(= 균형 완벽)는 **절대** 불가 — 200개 중 **0개**.
- 이 “**24.1**”이 정렬 기준: Ch. 2의 ϵ -greedy가 24를 크게 넘어서고, Ch. 9의 DQN이 **500에** 근접하는 과정을 **같은 숫자로** 따라가며 “배우기”가 실제로 일어나는 것을 확인.
- 시드 42: 24스텝 만에 쓰러짐 — 카트 위치 $x(t)$ 는 거의 제자리, 막대 $\theta(t)$ 는 20스텝경부터 급격히 벌어져 -0.2094 rad(약 -12°)를 넘고 terminated.



무작위 정책 24스텝 에피소드의 관측 변화 — 왼쪽: 카트 위치 $x(t)$, 오른쪽: 막대 각도 $\theta(t)$. ± 0.2094 rad(약 $\pm 12^\circ$)를 넘으면 terminated.

자주 묻는 질문

- **reset이 (obs, info)를 돌려주는데 info는?** Gymnasium 1.x는 **2-튜플**. info는 초기 관측과 함께 오는 부가 정보(샘플링 방식 등) — 교육 환경에서는 **빈 dict** {}. obs만 필요하면 obs, _ = env.reset(seed=...)로 _를 붙여 무시하는 것이 관례.
- **Gymnasium 1.x의 계약:** step은 **5-튜플**(obs, reward, terminated, truncated, info), reset은 **2-튜플**. 구버전 gym은 step이 4-튜플(done 하나) — 인터넷 예제에서 done만 나오는 4-튜플을 만나면, 이번 책에서는 terminated/truncated로 **나누어** 받는다는 점만 기억.
- **action_space.sample()의 무작위성은 어떻게 고정?** action_space의 무작위 발생기는 **상태를 유지** — 같은 시드를 주면 한 에피소드 내 **전체 행동 시퀀스**가 재현. 단 reset(seed=...)으로 **자동 고정되지 않아서** action_space.seed(7)를 **별도로** 부른다. 완전한 재현에는 **두 시드 모두** 고정.
- **로컬에 설치가 안 되면?** 이 책의 모든 노트북은 **Google Colab**에서 바로 실행(Colab 배지). Colab은 CPU만 주지만 CartPole·Pendulum 같은 **가벼운** 환경엔 충분. Ch. 13~14의 무거운 시뮬레이터는 Colab GPU 런타임.

1.3절의 한 줄

reset/step 두 함수가 “지금 상태를 보고, 하나를 행동하면 다음 상태·보상·종료 여부가 돌아온다”는 **유일한 계약**. 이 계약이 1.1절의 “관측 → 행동 → 보상 → 다음 관측” RL 정의를 **코드로 만든 것이다**.

이번 주차 정리

- ① **1.1 RL의 정체성** — 데이터(에이전트가 만든다) · 신호(정답 y 가 아니라 보상) · 시간(행동이 미래를 바꾼다) · 새 딜레마(탐험 vs 활용). **리턴** $G_t = \sum \gamma^k R_{t+k}$ 가 최대화 대상이고, 로봇 시뮬레이션은 연속 상태·행동 + 물리 법칙이라 **표 기반만으로는 안 된다**.
- ② **1.2 신경망 뼈대** — 활성 함수 없이는 2층이 선형 하나 (XOR로 검증), 역전파 = “오차를 한 층씩 거슬러 올려보낸다” (1-2-1 손계산). **새로움은 학습 알고리즘이 아니라 손실함수의 정의** — `zero_grad()` → `backward()` → `step()` 세 줄은 지도/강화 그대로.
- ③ **1.3 Gymnasium** — `reset/step` 계약, `terminated`(진짜 종료) vs `truncated`(시간 한도), 시드 두 개를 모두 고정하는 재현성. **무작위 정책 베이스라인 = 평균 24.1스텝** — 이후 모든 알고리즘의 “성공”을 잴 기준.

다음 주차 — Chapter 2. 멀티암 밴딧

상태도, 전이도 없는 **가장 단순한** RL 문제에서 “탐험 vs 활용” 딜레마를 **순수한** 형태로 만난다. ϵ -greedy와 UCB로 처음 풀어보며 — CartPole의 24.1스텝을 넘는 것이 첫 번째 관문이다.